

Development process

How Jarvis was built and how to keep changing it safely: phases, update kits, git, tests, continuous integration and recipes for common changes.

Jarvis 1.0, October 2026

In this document

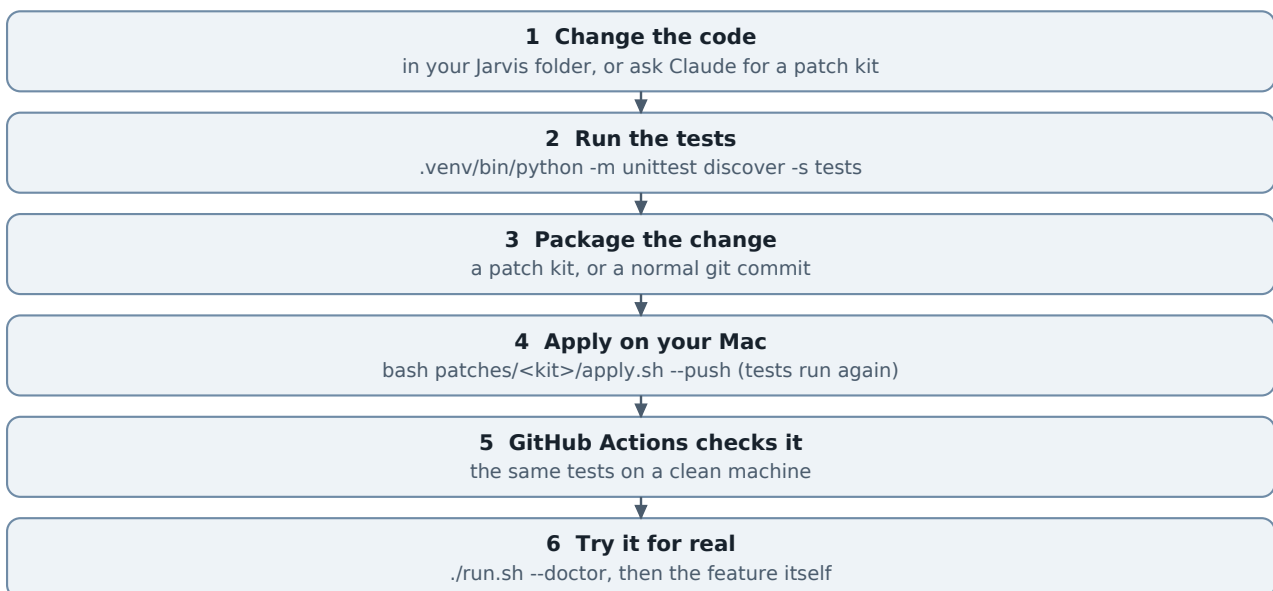
- How the project grew: ten phases
- The development loop
- Update kits: what's inside, what `apply.sh` does
- Git and GitHub
- Testing
- Recipes: add a tool, a setting, a quick phrase, change the personality or the look
- Versions and the change checklist

How the project grew

Phase	Version	What it added	Tests
1. Foundation	0.1.0	Installer, settings, the doctor with a live Claude test, GitHub CI	23
2. Brain	0.2.0	Agent loop, streaming replies, web search, memory, terminal chat	49
3. Files and terminal	0.3.0	File tools, the shell tool, safety rules, backups, patch kits	64
4. Mac control	0.4.0	Opening things, AppleScript, clipboard, typing into apps, notifications	83
5. Ears	0.5.0	Recorder, Whisper, --listen, --voice	100
6. Voice	0.6.0	Spoken replies, the assistant engine, quick phrases, follow-ups	118
7. Menu-bar app	0.7.0	Status icon, menu, global hotkeys, hold-to-talk	140
8. Floating panel	0.8.0	The glass panel with the animated orb	149
9. Tutor mode	0.9.0	Screenshots, the pointer, /tutor	161
10. Polish	1.0.0	Wake word, start at login, the final doctor	172

Each phase was a complete, tested step that left Jarvis working, so problems surfaced one layer at a time.

The development loop



- **Small steps.** Change one thing, run the tests, then try it. When something breaks, you know which change did it.
- **Tests first for logic.** Anything that isn't drawing or talking to macOS can be tested on any computer.
- **macOS code twice.** AppKit and Quartz code is tested against stand-ins, then tried on the Mac.

Update kits

What's inside

Part	Purpose
<code>apply.sh</code>	The script that applies the update (see below).
<code>changes.patch</code>	A git patch from your repository's exact commit to the new version.
<code>files/</code>	Complete copies of every project file, used when the patch doesn't fit.
<code>files.txt</code>	The list of project files.
<code>removed.txt</code>	Files the update removes (each is backed up first).
<code>CHANGES.txt</code>	What's new, and which files were added, modified or removed.

What `apply.sh` does

1. Finds your Jarvis folder: the one you're in, or the one the kit was unzipped into. It reads the current phase from `jarvis/__init__.py` and refuses to go backwards.
2. Applies the patch with `git apply` when it fits cleanly. Otherwise it copies each changed file, first saving your version in `patches/backups/before-phase-N-<date-time>/`.
3. Moves leftovers out of the way: folders like `jarvis 2`, old `jarvis-phase-*.zip` files and removed files.
4. Runs `./setup.sh` if there's no `.venv`, or reinstalls packages if `requirements.txt` changed.
5. Runs every test. If one fails, it stops and commits nothing; the output is in `data/logs/patch-phase-N-tests.log`.
6. With `--commit` it commits as “Phase N: ...”; with `--push` it also pushes.
7. Removes its own folder from `patches/` (keep it with `--keep`).

Unzip kits inside your Jarvis folder, or run `apply.sh` from inside it. Copying a zip's folder into the repository by hand is what created the stray `jarvis 2` folder during phase 2.

Git and GitHub

- Everything lives on `main`; each phase is one commit with a clear message.
- `.gitignore` keeps out `.env`, `config.local.yaml`, `Jarvis.command`, `.venv/`, `data/`, `patches/` and `caches`. Never commit your API key; if one ever leaks, delete it in the Claude Console and create a new one.
- GitHub Actions (`.github/workflows/tests.yml`) runs on every push: Python 3.12 on Ubuntu with `actions/checkout@v6` and `actions/setup-python@v6`. It installs the test packages, checks syntax (`compileall` and `bash -n`) and runs the unit tests. The Actions tab should show green.
- To undo a change: `git revert <commit>`. To look at an older version: `git checkout <commit>`, then `git checkout main`.
- To find which commit broke something: `git bisect start`, mark a bad and a good commit, and run the tests at each step.

Testing

Every part of Jarvis is tested without real hardware or a real API, so the tests are fast (about 7 seconds) and safe to run anytime.

Test file	Covers
test_foundation.py	Settings, API error messages, the doctor, the command line
test_brain.py	The agent loop, history, memory, the tool registry, prompts, terminal chat
test_tools.py	File and shell tools, safety rules, writing and running a script
test_mac.py	Mac control tools and the terminal typing guard
test_voice.py	The recorder, the Whisper wrapper, talking in the terminal
test_assistant.py	Spoken replies and the assistant engine
test_app.py	Hotkeys, the menu-bar app and the menu status line
test_panel.py	The floating panel and its orb
test_tutor.py	Screenshots, the pointer and the tutor flow
test_finish.py	The wake word, start at login and the final checks

- **Claude API:** answered by `FakeAPI` in the real streaming format, built with the `reply()` helper.
- **Microphone:** `FakeMicrophone` plays scripted loudness about 100 times faster than real time.
- **Whisper, openWakeWord, AppKit and Quartz:** replaced by small stand-in modules.
- **Nothing real happens:** no recording, speech, downloads, notifications or menu-bar changes.

```
.venv/bin/python -m unittest discover -s tests -v          # everything
.venv/bin/python -m unittest tests.test_voice -v          # one file
.venv/bin/python -m unittest tests.test_tools.ShellTests -v # one group
```

Recipes

Add a tool

1. Write a function `my_tool(ctx, args)` in a module under `jarvis/tools/`. Raise `ToolError("...")` for problems Claude should see, call `ctx.check_cancel()` in long loops, and `ctx.confirm(title, message)` before anything risky.
2. Register it in the module's `register(reg)` with `reg.add(name, description, properties, required=..., func=..., status=..., activity=...)`.
3. For a new module, add its name to `TOOL_MODULES` in `jarvis/tools/__init__.py`.
4. Add guidance to `TOOL_NOTES` in `prompts.py`, keyed by the tool's name, so it only appears when the tool exists.
5. Add tests: call it through `build_registry(cfg).execute(...)`, plus an end-to-end test with `FakeAPI`.

A complete small example, a battery tool:

```
import subprocess

from . import ToolContext

def battery(ctx: ToolContext, args: dict) -> str:
    out = subprocess.run(["pmset", "-g", "batt"], capture_output=True, text=True, timeout=10).stdout
    return out.strip() or "No battery information."

def register(reg) -> None:
    reg.add("battery", "Battery level and whether the Mac is charging.", {}, func=battery,
           status="Checking the battery", activity="Checking the battery...")
```

Add a setting

1. Add it to `DEFAULTS` in `config.py`; if it's a number, also to `_NUMBERS` with its minimum.
2. Add the same key, value and a comment to `config.yaml`. A test checks that `config.yaml` matches `DEFAULTS` exactly.
3. Read it in code as `cfg.section.key`, and mention it in the README.

Add a quick phrase

Edit the phrase sets at the top of `assistant.py` (`RESET_PHRASES`, `NEVERMIND_PHRASES`, `REPEAT_PHRASES`, `THANKS_PHRASES`) or add a branch to `_local_command`.

Change the personality or the rules

Edit `IDENTITY`, `REPLIES`, `PRINCIPLES` and `TEACHING` in `prompts.py`. Keep the rules for spoken replies short and plain, and run the prompt tests in `test_brain.py`.

Change the look

Panel colours, titles and size: `STATE_COLORS`, `TITLES`, `WIDTH` and `HEIGHT` in `ui/hud_logic.py`. Pointer: `AMBER` and `BUBBLE_BG` in `ui/overlay.py`, and `ARROW` and `MOVE_SECONDS` in `ui/overlay_logic.py`.

Versions and the change checklist

The version and phase live in `jarvis/__init__.py` (`__version__ = "1.0.0"`, `PHASE = 10`); `./run.sh --version` and the doctor show them.

Before shipping any change:

- The tests pass on your Mac and on GitHub.
- Anything that changes files, runs commands or acts in apps is covered by a safety rule or asks first.
- New settings are in both `DEFAULTS` and `config.yaml`.
- The prompt only describes tools that exist.
- Screen updates go through the UI bridge, never straight from the worker thread.
- The README and these documents still match what Jarvis does.